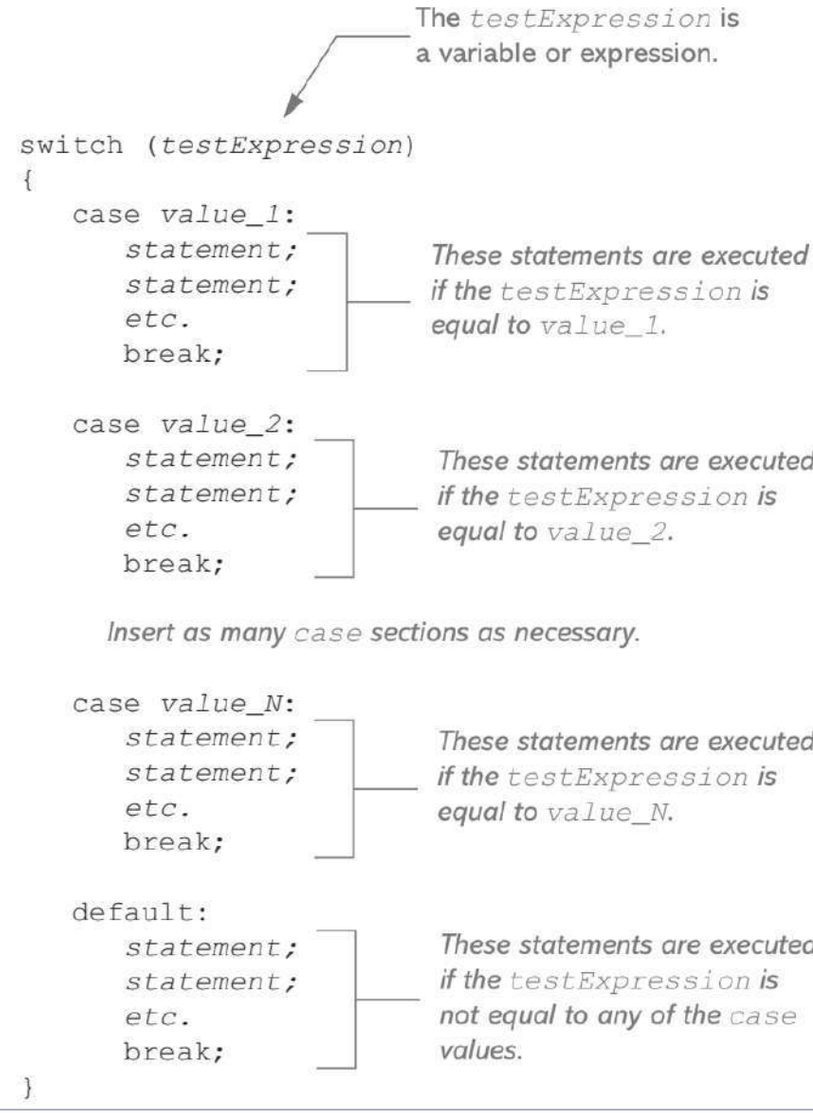
Switch Statement



Switch Statement

- Switch statement works like this if-else-if statement:

```
switch (month)
{
    case 1:
        MessageBox.Show("January");
        break;

    case 2:
        MessageBox.Show("February");
        break;

    case 3:
        MessageBox.Show("March");
        break;

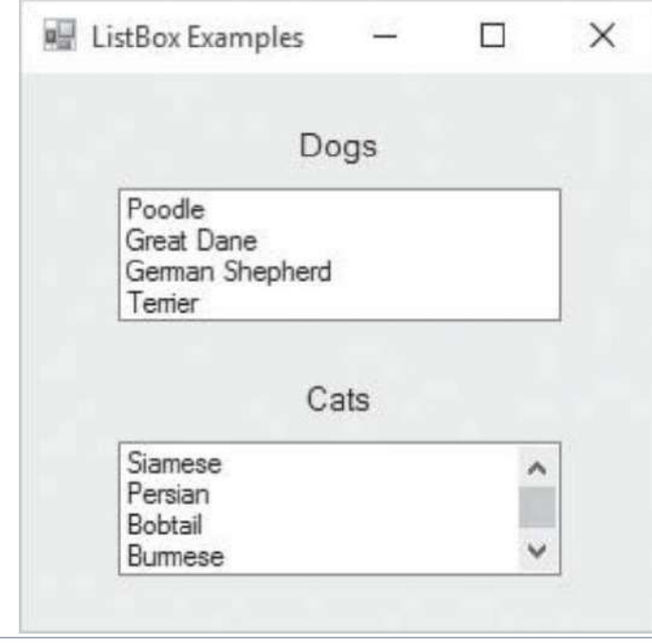
    default:
        MessageBox.Show("Error: Invalid month");
        break;
}
```

```
if (month == 1)
{
    MessageBox.Show("January");
}

else if (month == 2)
{
    MessageBox.Show("February");
}
else if (month == 3)
{
    MessageBox.Show("March");
}
else
{
    MessageBox.Show("Error: Invalid month");
}
```

List Boxes

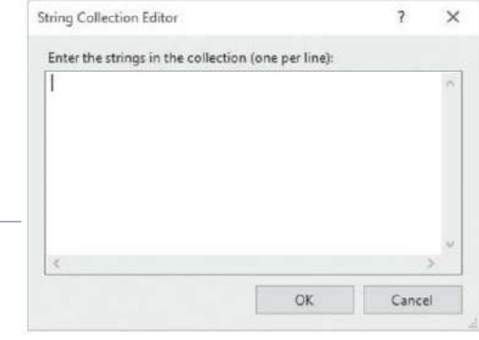
- List boxes display a list of items and allow the user to select an item from the list.
- At run time, the user may select one of the items, causing the item to appear selected.
- The topmost ListBox in the Figure does not have a scroll bar, but the bottom one does.
- A scroll bar appears when a ListBox contains more items than can be displayed in the space provided.



List Boxes

- Once you create a ListBox control, you add items to its Items property.
- The items that you add to a ListBox's Items property are displayed in the ListBox.
- To store values in the Items property at design time, follow these steps:

1. Select the ListBox control in the Designer window.
2. In the Properties window, the setting for the Items property is displayed as (Collection). When you select the Items property, an ellipses button (...) appears.
3. Click the ellipses button. The String Collection Editor dialog box appears, as shown in the Figure.
4. Type the values that are to appear in the ListBox into the String Collection Editor dialog box. Type each value on a separate line by pressing the Enter key after each entry.
5. When you have entered all the values, click the OK button.



SelectedItem Property

- When the user selects an item in a ListBox, the item is stored in the ListBox's SelectedItem property.
- For example, suppose an application has a ListBox control named fruitListBox and a string variable named selectedFruit.
- The fruitListBox control contains the items Apples, Pears, and Bananas.
- If the user has selected Pears, the following statement assigns the string "Pears" to the variable selectedFruit:

```
selectedFruit = fruitListBox.SelectedItem.ToString();
```

Determining Whether an Item Is Selected

- An exception will occur if you try to get the value of a ListBox's SelectedItem property when no item is selected in the ListBox.
- For that reason, you should always make sure that an item is selected before reading the SelectedItem property.
- You do this with the SelectedIndex property.

SelectedIndex Property

- The items that are stored in a ListBox each have an index.
- The index is simply a number that identifies the item's position in the ListBox.
- The first item has the index 0, the next has the index 1, and so forth.
- The last index value is n − 1, where n is the number of items in the ListBox.
- When the user selects an item in a ListBox, the item's index is stored in the ListBox's SelectedIndex property.
- If no item is selected in the ListBox, the SelectedIndex property is set to −1.

SelectedIndex Property

- You can use the SelectedIndex property to make sure that an item is selected in a ListBox before you try to get the value of the SelectedItem property.
- **Example:**

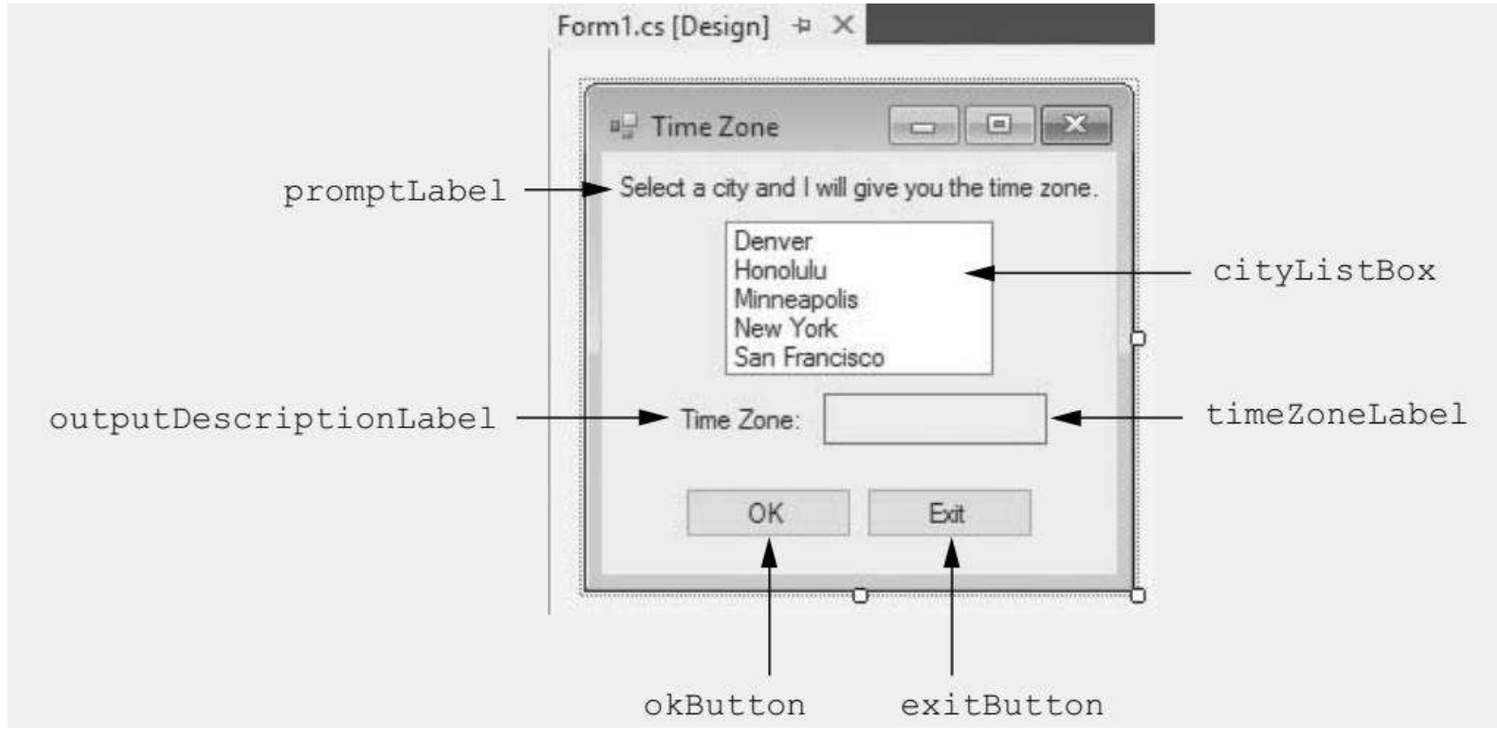
```
if (fruitListBox.SelectedIndex != −1)
{
selectedFruit = fruitListBox.SelectedItem.ToString();
}
```

Example 9

Time Zone application

Create an application that allows the user to select a city from a ListBox control.

When the user clicks a button, the application displays the name of the city's time zone.



Example 9- Control property settings

Control Name	Control Type	Property Settings
`promptLabel`	Label	**Text:** *Select a city and I will give you the time zone.*
`cityListBox`	ListBox	**Items:** *Denver* *Honolulu* *Minneapolis* *New York* *San Francisco*
`outputDescriptionLabel`	Label	**Text:** *Time Zone:*
`timeZoneLabel`	Label	**AutoSize:** False **BorderStyle:** FixedSingle **Text:** (The contents of the Text property have been erased.) **TextAlign:** MiddleCenter
`okButton`	Button	**Text:** *OK*
`exitButton`	Button	**Text:** *Exit*

Example 9- Form1 code for Time Zone application

```
private void okButton_Click(object sender, EventArgs e)
{
    string city;   // To hold the name of a city

    if (cityListBox.SelectedIndex != -1)
    {
        // Get the selected item.
        city = cityListBox.SelectedItem.ToString();
        // Determine the time zone.
        switch (city)
        {
            case "Honolulu":
                timeZoneLabel.Text = "Hawaii-Aleutian";
                break;
            case "San Francisco":
                timeZoneLabel.Text = "Pacific";
                break;
            case "Denver":
                timeZoneLabel.Text = "Mountain";
                break;
            case "Minneapolis":
                timeZoneLabel.Text = "Central";
                break;
            case "New York":
                timeZoneLabel.Text = "Eastern";
                break;
        }
    }
    else
    {
        // No city was selected.
        MessageBox.Show("Select a city.");
    }
}
```

ListBoxes

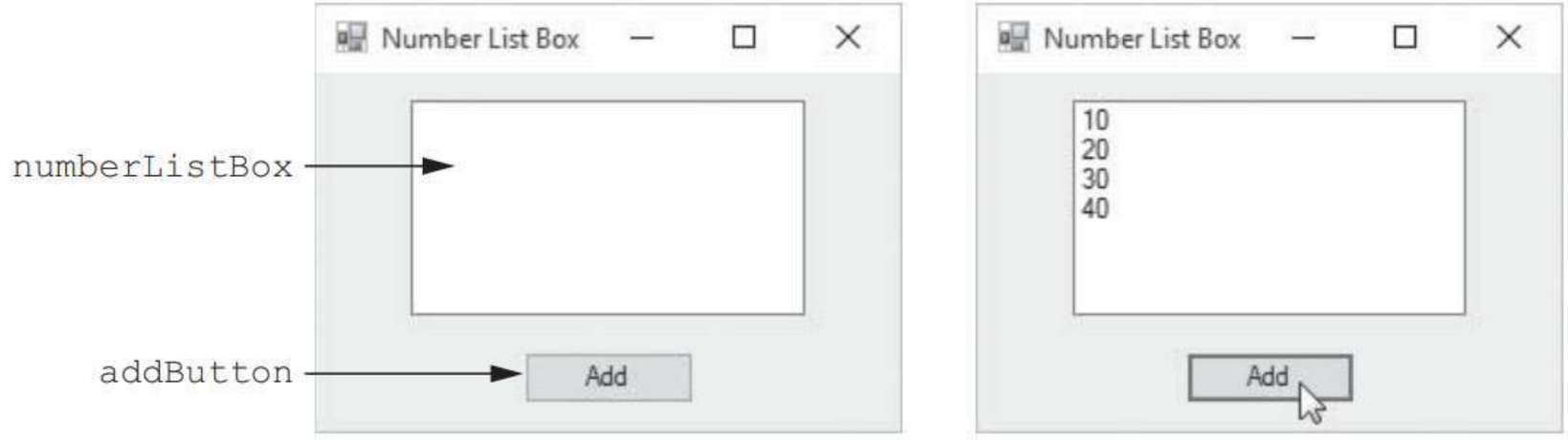
- Previously, we introduced the ListBox control, which displays a list of items and allows the user to select one or more items from the list.
- Previously, you add items to a ListBox control's Items property, and those items are displayed in the ListBox.
- Now, we use ListBox controls to display output.
- At design time, you can use the Properties window to add items to the control's Items property.
- You can also write code that adds items to a ListBox control at run time.
- To add an item to a ListBox control with code, you call the control's Items.Add method.

```
ListBoxName.Items.Add(Item);
```

Example 10

Number List application

When you click the addButton control, the numbers shown in the image on the right are added to the numberListBox control.



Example 10- Form1 code for Number List application

```
1 private void addButton_Click(object sender, EventArgs e)
2 {
3       numberListBox.Items.Add(10);
4       numberListBox.Items.Add(20);
5       numberListBox.Items.Add(30);
6       numberListBox.Items.Add(40);
7 }
```

Items.Count Property

- ListBox controls have an Items.Count property that reports the number of items stored in the ListBox.
- If the ListBox is empty, the Items.Count property equals 0.
- For example, assume an application has a ListBox control named employeesListBox. The following if statement displays a message box if there are no items in the ListBox:

```
if (employeesListBox.Items.Count == 0)
{
MessageBox.Show("There are no items in the list!");
}
```

- The Items.Count property holds an integer value.
- Assuming numEmployees is an int variable, the following statement assigns the number of items in the employeesListBox to the numEmployees variable:

```
numEmployees = employeesListBox.Items.Count;
```

Items.Clear Method

- ListBox controls have an Items.Clear method that erases all the items in the Items property.

```
ListBoxName.Items.Clear();
```

- For example, assume an application has a ListBox control named employeesListBox.
- The following statement clears all the items in the list:

```
employeesListBox.Items.Clear();
```

Items.RemoveAt

- The method RemoveAt is used to remove an item from the ListBox

```
ListBox.Items.RemoveAt(Index);
```

Example 11

Adds, removes and clears ListBox items Application

When the user clicks the Add Button, the new item appears in displayListBox.

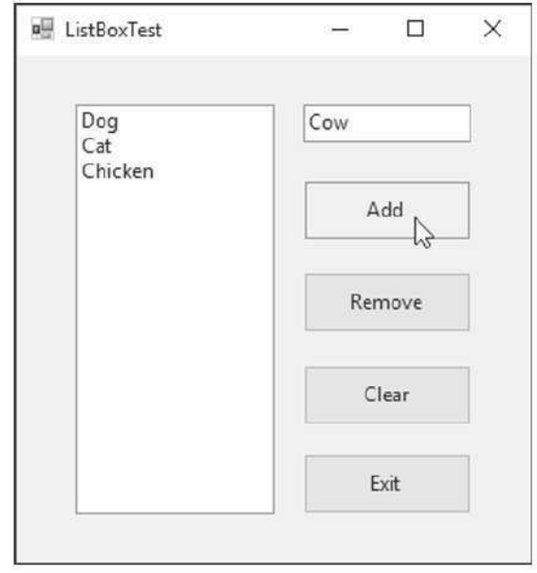
Similarly, if the user selects an item and clicks Remove, the item is deleted.

When clicked, Clear deletes all entries in displayListBox.

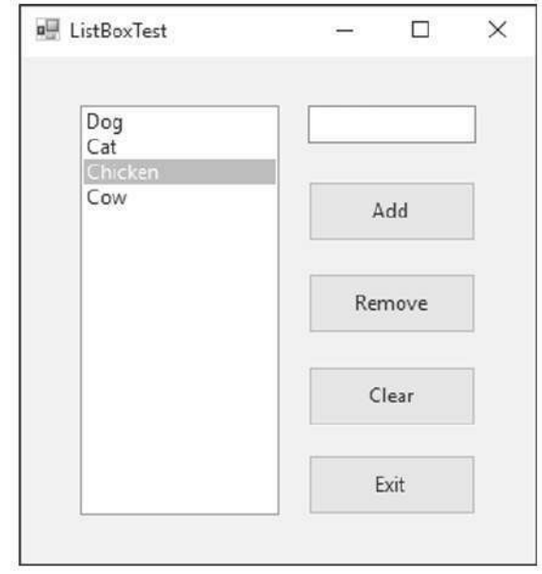
The user terminates the app by clicking Exit.

Example 11

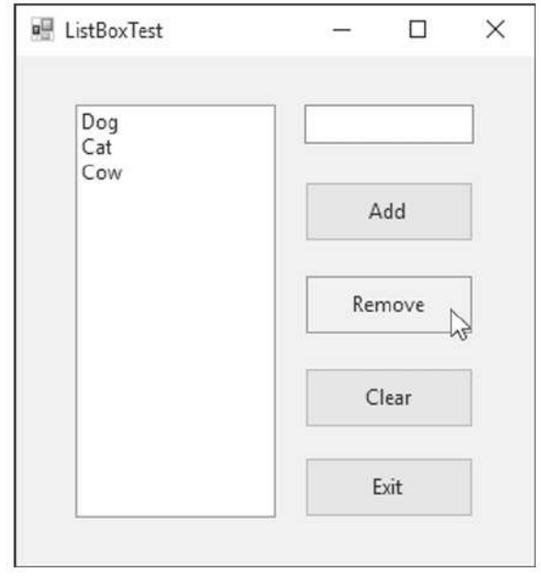
a) GUI after adding **Dog**, **Cat** and **Chicken** and before adding **Cow**



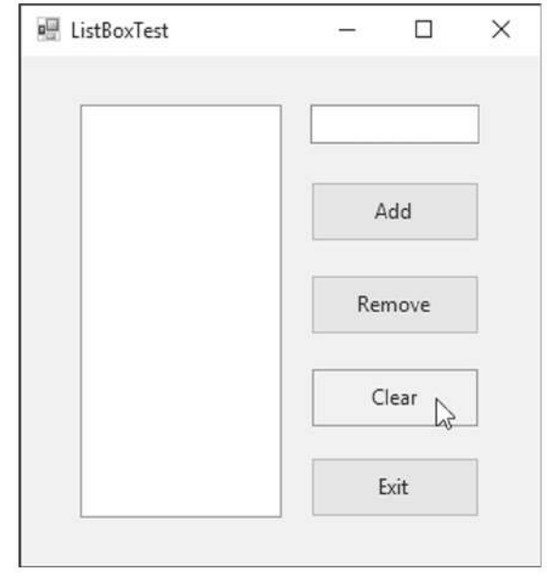
b) GUI after adding **Cow** and before deleting **Chicken**



c) GUI after deleting **Chicken**



d) GUI after clearing the `ListBox`



Example 11- Form1 code for Adds, removes and clears ListBox items Application

```
// add new item to ListBox (text from input TextBox)
// and clear input TextBox
private void addButton_Click(object sender, EventArgs e)
{
   displayListBox.Items.Add(inputTextBox.Text);
   inputTextBox.Clear();
}

// remove item if one is selected
private void removeButton_Click(object sender, EventArgs e)
{
   // check whether item is selected; if so, remove
   if (displayListBox.SelectedIndex != -1)
   {
      displayListBox.Items.RemoveAt(displayListBox.SelectedIndex);
   }
}

// clear all items in ListBox
private void clearButton_Click(object sender, EventArgs e)
{
   displayListBox.Items.Clear();
}
```


THANK YOU

Chapter 4

Loops, Files, and random numbers

By

Dr. Sara Tarik Kamal

While Loop

- The while loop causes a statement or set of statements to repeat as long as a Boolean expression is true.

```
while (BooleanExpression)
{
      statement;
      statement;
      etc.
}
```

This set of statements is repeated while the Boolean expression is true.

- If you are writing a while loop that has only one statement in its body, you do not have to enclose the statement inside curly brace

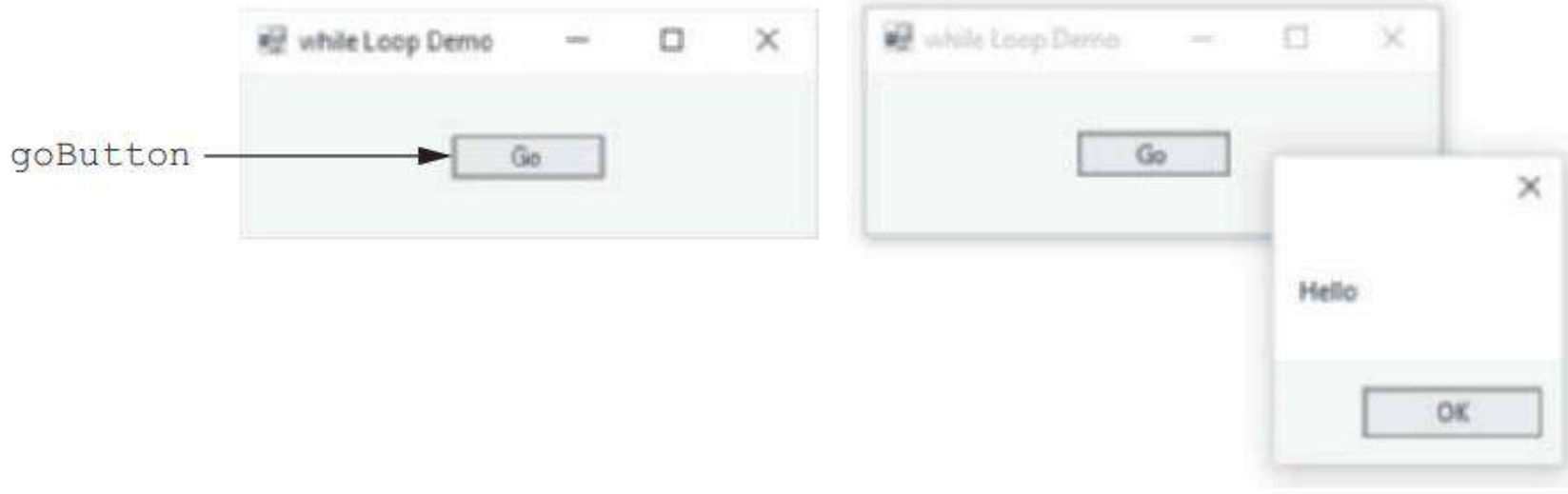
```
while (BooleanExpression)
      statement;
```

Example 1

While Loop application

When you click the OK button to close the message box, another identical message box is displayed.

The message box is displayed a total of five times.



Example 1- Form1 code for While Loop application

```
1  private void goButton_Click(object sender, EventArgs e)
2  {
3      // Declare a variable to count the loop iterations.
4      int count = 1;
5
6      // Display "Hello" in a message box five times.
7      while (count <= 5)
8      {
9          // Display the message box.
10         MessageBox.Show("Hello");
11
12         // Add one to count.
13         count = count + 1;
14     }
15 }
```

While Loop (Pretest Loop)

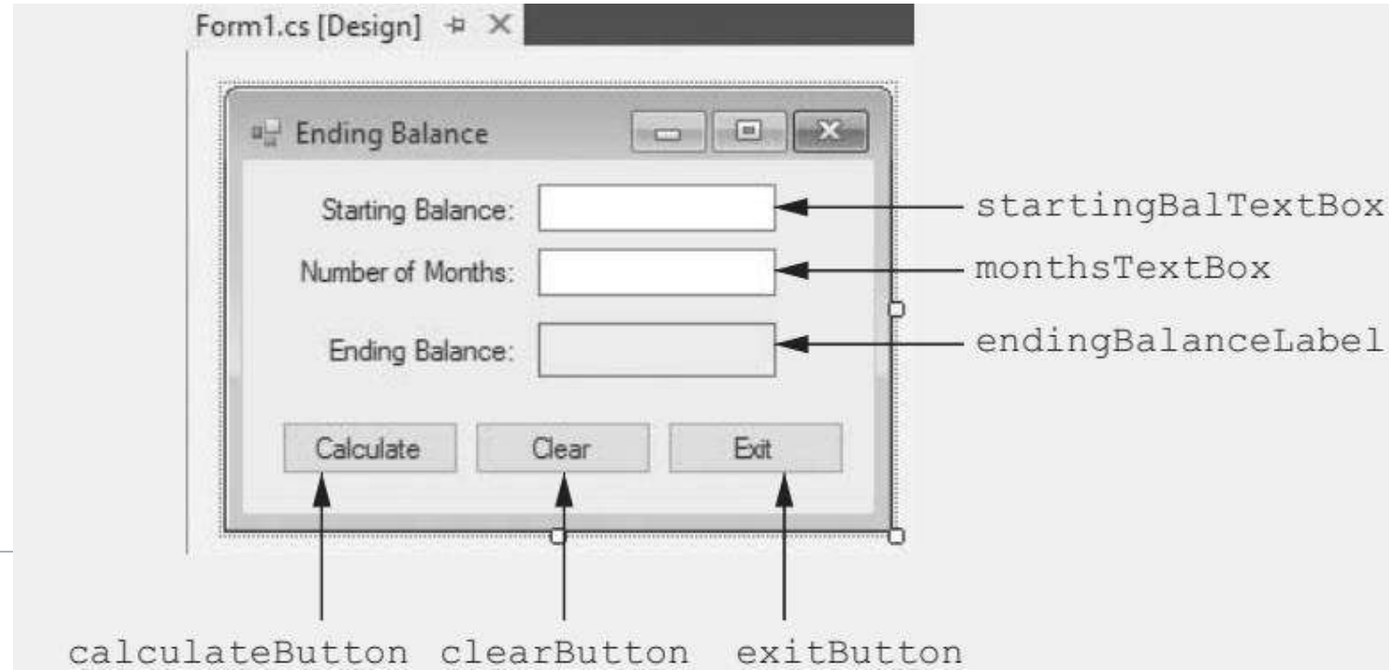
- The while loop is known as a pretest loop, which means it tests its condition before performing an iteration.
- Because the test is done at the beginning of the loop, you usually have to perform some steps prior to the loop to make sure that the loop executes at least once.

```
1  private void goButton_Click(object sender, EventArgs e)
2  {
3      // Declare a variable to count the loop iterations.
4      int count = 6;
5
6      // This loop will never iterate!
7      while (count <= 5)
8      {
9          // Display the message box.
10         MessageBox.Show("Hello");
11
12         // Add one to count.
13         count = count + 1;
14     }
15 }
```

Example 2

Account Balance Application

Allow the user to enter an account's starting balance into the startingBalTextBox control and the number of months that the account will be left to earn interest into the monthsTextBox control. When the user clicks the calculateButton control, the application calculates the account's balance at the end of the time period. The account's monthly interest rate is 0.005, and the interest is compounded monthly.



Example 2- Form1 code for Account Balance Application

```
private void calculateButton_Click(object sender, EventArgs e)
{
    // Constant for the monthly interest rate.
    const decimal INTEREST_RATE = 0.005m;

    // Local variables
    decimal balance;     //  The account balance
    int months;          //  The number of months
    int count = 1;       //  Loop counter, initialized with 1

    // Get the starting balance.
    if (decimal.TryParse(startingBalTextBox.Text, out balance))
    {
        // Get the number of months.
        if (int.TryParse(monthsTextBox.Text, out months))
        {
            // The following loop calculates the ending balance.
            while (count <= months)
            {
                // Add this month's interest to the balance.
                balance = balance + (INTEREST_RATE * balance);

                // Add one to the loop counter.
                count = count + 1;
            }

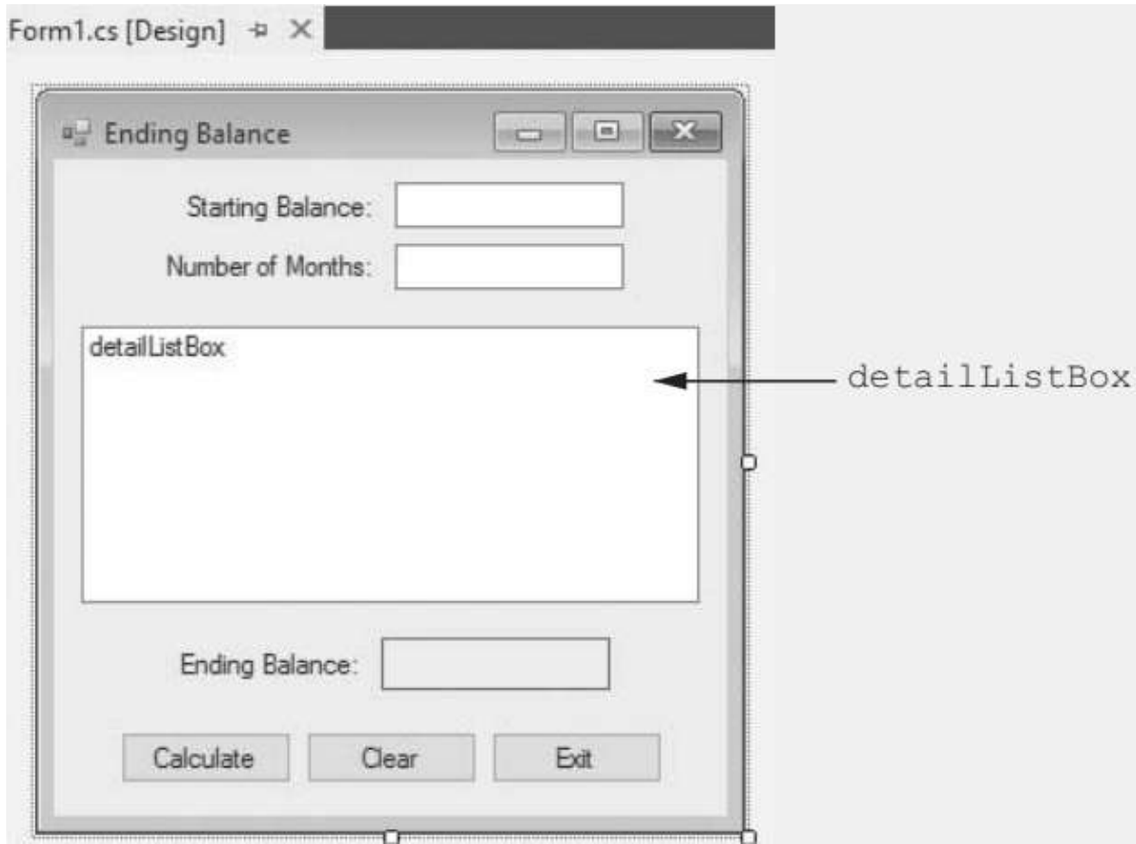
            // Display the ending balance.
            endingBalanceLabel.Text = balance.ToString("c");
        }
        else
        {
            // Invalid number of months was entered.
            MessageBox.Show("Invalid value for months.");
        }
    }
    else
    {
        // Invalid starting balance was entered.
        MessageBox.Show("Invalid value for starting balance.");
    }
}
```

Example 3

Enhance Account Balance Application

First, add a ListBox control to the application's form.

Then modify the calculateButton_Click event handler so it displays each month's ending balance in the ListBox.





Example 3- Form1 code for Enhance Account Balance Application

```
private void calculateButton_Click(object sender, EventArgs e)
{
    // Constant for the monthly interest rate.
    const decimal INTEREST_RATE = 0.005m;

    // Local variables
    decimal balance;     // The account balance
    int months;          // The number of months
    int count = 1;       // Loop counter, initialized with 1

    // Get the starting balance.
    if (decimal.TryParse(startingBalTextBox.Text, out balance))
    {
        // Get the number of months.
        if (int.TryParse(monthsTextBox.Text, out months))
        {
            // The following loop calculates the ending balance.
            while (count <= months)
            {
                // Add this month's interest to the balance.
                balance = balance + (INTEREST_RATE * balance);

                // Display this month's ending balance.
                detailListBox.Items.Add("The ending balance " +
                    "for month " + count + " is " +
                    balance.ToString("c"));

                // Add one to the loop counter.
                count = count + 1;
            }

            // Display the ending balance.
            endingBalanceLabel.Text = balance.ToString("c");
        }
        else
        {
            // Invalid number of months was entered.
            MessageBox.Show("Invalid value for months.");
        }
    }
    else
    {
        // Invalid starting balance was entered.
        MessageBox.Show("Invalid value for starting balance.");
    }
}
```

for Loop

- When you write a for loop, you specify three actions:
  - Initialization: This action takes place when the loop begins. It happens only once.
  - Test: A Boolean expression is tested. If the expression is true, the loop iterates. Otherwise, the loop stops.
  - Update: This action takes place at the end of each loop iteration

```
for (InitializationExpression; TestExpression; UpdateExpression)
{
    statement;
    statement;
    etc.
}
```

- If the body of the loop contains only one statement:

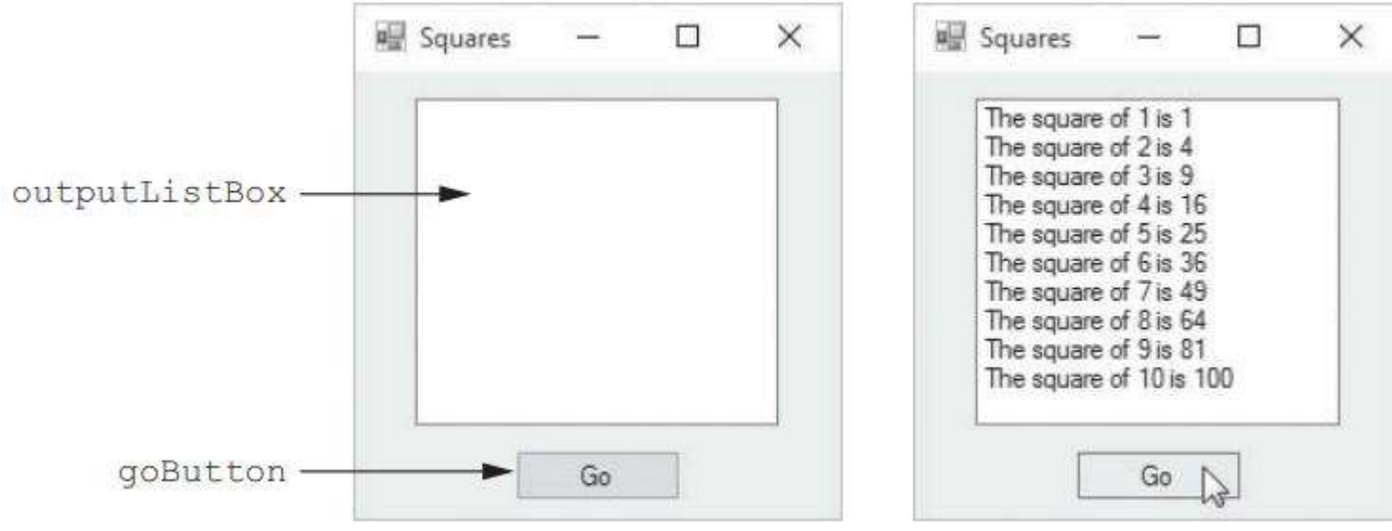
```
for (InitializationExpression; TestExpression; UpdateExpression)
   statement;
```

Example 4

Squares application

The application display the numbers 1–10 and their squares.

At run time, when you click the goButton control, the outputListBox control displays the program's output.



Example 4- Form1 code for Squares application

```
1  private void goButton_Click(object sender, EventArgs e)
2  {
3      // Constant for the maximum number
4      const int MAX_VALUE = 10;
5
6      // Loop counter
7      int number;
8
9      // Display the list of numbers and their squares.
10     for (number = 1; number <= MAX_VALUE; number++)
11     {
12         outputListBox.Items.Add("The square of " +
13             number + " is " + (number * number));
14     }
15 }
```

for Loop (Pretest Loop)

- Because the for loop tests its Boolean expression before it performs an iteration, it is a pretest loop.
- It is possible to write a for loop in such a way that it will never iterate.
- Example:

```
for (count = 6; count <= 5; count++)
{
   MessageBox.Show("Hello");
}
```

- Because the variable count is initialized to a value that makes the Boolean expression false from the beginning, this loop terminates as soon as it begins.

Declaring the Counter Variable in the Initialization Expression

- Not only may the counter variable be initialized in the initialization expression, but it may also be declared there.

```
for (int count = 1; count <= 5; count++)
{
   MessageBox.Show("Hello");
}
```

- When a variable is declared in the initialization expression of a for loop, the scope of the variable is limited to the loop.
- This means you cannot access the variable in statements outside the loop.
- For example, the following code would cause a compiler error because the statement in line 6 cannot access the count variable.

```
1 for (int count = 1; count <= 5; count++)
2 {
3     MessageBox.Show("Hello");
4 }
5
6 MessageBox.Show("The value of count is " + count);
```

Example 5

Speed Converter application

Create an application that displays a table of speeds in kilometers per hour with their values converted to miles per hour.

The formula for converting kilometers per hour to miles per hour is:

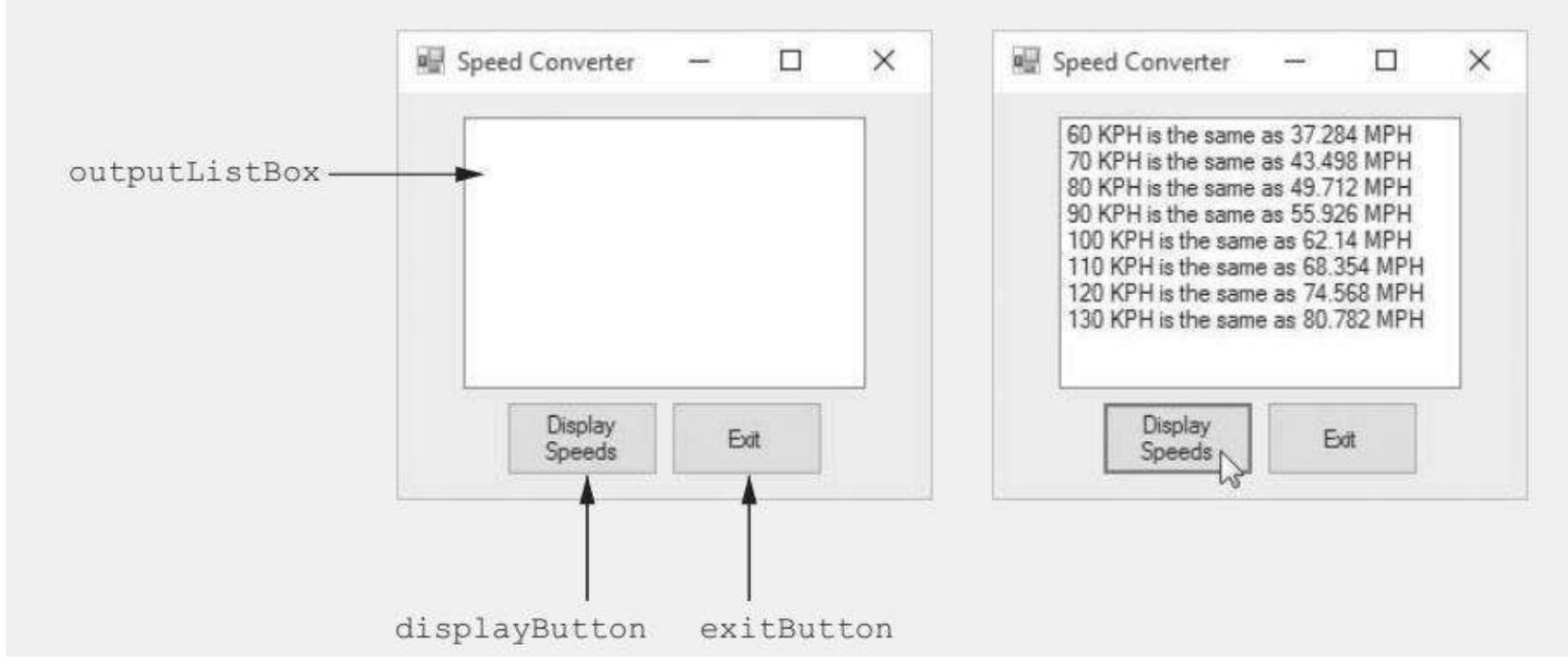
MPH = KPH * 0.6214

In the formula, MPH is the speed in miles per hour and KPH is the speed in kilometers per hour. The table that your program displays should show speeds from 60 kilometers per hour through 130 kilometers per hour, in increments of 10, along with their values converted to miles per hour. The table should look something like this:

KPH	MPH
60	37.284
70	43.498
80	49.712
etc. . . .	

Example 5- Speed Converter form

- The application's form, at run time, is shown in the Figure. The image on the left in the figure shows the names of the controls. The image on the right shows how the form appears after the user clicks the Display Speeds button.



Example 5- Form1 code for Speed Converter application

```
private void displayButton_Click(object sender, EventArgs e)
{
    // Constants
    const int START_SPEED = 60;
    const int END_SPEED = 130;
    const int INTERVAL = 10;
    const double CONVERSION_FACTOR = 0.6214;

    // Variables
    int kph;         // Kilometers per hour
    double mph;      // Miles per hour

    // Display the table of speeds.
    for (kph = START_SPEED; kph <= END_SPEED; kph += INTERVAL)
    {
        // Calculate miles per hour.
        mph = kph * CONVERSION_FACTOR;

        // Display the conversion.
        outputListBox.Items.Add(kph + " KPH is the same as " +
            mph + " MPH");
    }
}

private void exitButton_Click(object sender, EventArgs e)
{
    // Close the form.
    this.Close();
}
```

do-while Loop

- The do-while loop is a posttest loop, which means it performs an iteration before testing its Boolean expression.

```
do
{
    statement;
    statement;
    etc.
} while (BooleanExpression);
```

- The general format of the do-while loop with only one conditionally executed statement is:

```
do
    statement;
while (BooleanExpression);
```

Using Files for Data Storage

- The programs you have written so far require the user to reenter data each time the program runs because data kept in variables and control properties is stored in RAM and disappears once the program stops running.
- If a program is to retain data between the times it runs, it must have a way of saving it.
- Data is saved in a file, which is usually stored on a computer's disk.
- Once the data is saved in a file, it will remain there after the program stops running.
- Data that is stored in a file can be retrieved and used at a later time.

Using Files for Data Storage

- The process of retrieving data from a file is known as reading data from the file.
- When a piece of data is read from a file, it is copied from the file into a variable in RAM.
- The term input file is used to describe a file from which data is read.
- It is called an input file because the program gets input from the file.

Write data to files and read data from files

- There are always three steps that must be taken when a file is used by a program.

1. Open the file—Opening a file creates a connection between the file and the program.
    - Opening an output file usually creates the file on the disk and allows the program to write data to it.
    - Opening an input file allows the program to read data from the file.
2. Process the file—In this step, data is either written to the file (if it is an output file) or read from the file (if it is an input file).
3. Close the file—When the program is finished using the file, the file must be closed. Closing a file disconnects the file from the program.

Types of Files

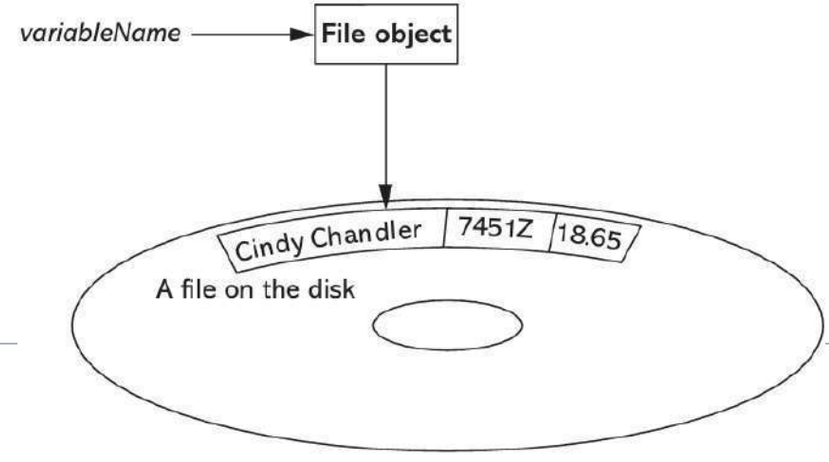
- In general, there are two types of files: text and binary.
- A text file contains data that has been encoded as text.
- Even if the file contains numbers, those numbers are stored in the file as a series of characters.
- As a result, the file may be opened and viewed in a text editor such as Notepad.
- A binary file contains data that has not been converted to text.
- As a consequence, you cannot view the contents of a binary file with a text editor.
- In this chapter, we work only with text files.

File Access Methods

- Most programming languages provide two different ways to access data stored in a file: sequential access and direct access.
- When you work with a sequential access file, you access data from the beginning of the file to the end of the file.
  - If you want to read a piece of data that is stored at the very end of the file, you have to read all the data that comes before it—you cannot jump directly to the desired data.
- When you work with a direct access file (which is also known as a random access file), you can jump directly to any piece of data in the file without reading the data that comes before it.
- This chapter focuses on sequential access files.

File Objects

- In order for a program to work with a file on the computer's disk, the program must create a file object in memory.
- A file object is an object that is associated with a specific file and provides a way for the program to work with that file.
- In the program, a variable is linked with the file object.
- We say that the variable references the object.
- This variable is used to carry out any operations that are performed on the file.



File Objects

- You will be using two classes from the .NET Framework to create file objects.
- When you want to write data to a text file, you use the StreamWriter class, and when you want to read data from a text file you use the StreamReader class.
- These classes are in the System.IO namespace in the .NET Framework, so you will need to write the following directive at the top of your programs:

```
using System.IO;
```

Writing Data to a File with a StreamWriter Object

- Suppose you want to write a program that creates a text file named courses.txt and writes the names of the courses you are taking to the file.
- The following code sample shows how you can do this:

```
1 StreamWriter outputFile;
2 outputFile = File.CreateText("courses.txt");
3
4 outputFile.WriteLine("Intro to Computer Science");
5 outputFile.WriteLine("English Composition");
6 outputFile.WriteLine("Calculus I");
7 outputFile.WriteLine("Music Appreciation");
8
9 outputFile.Close();
```

Writing Data to a File with a StreamWriter Object

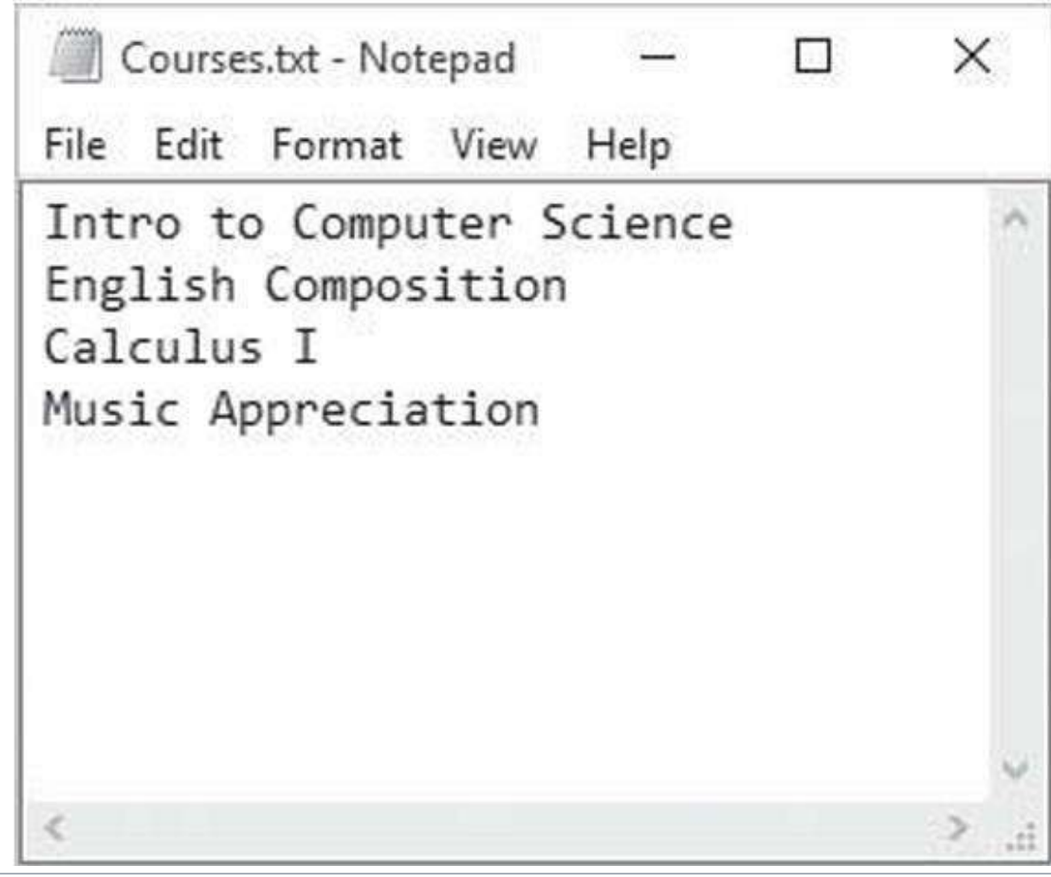
- **Line 1:** This statement declares a variable named outputFile, which can be used to reference a StreamWriter object.
- **Line 2:** In a nutshell, this statement opens the file to which you will be writing data. It does so by calling the File.CreateText method, passing the string "courses.txt" as an argument. The File.CreateText method does the following:
  - It creates a text file with the name specified by the argument. If the file already exists, its contents are erased.
  - It creates a StreamWriter object in memory, associated with the file.
  - It returns a reference to the StreamWriter object.
    - Notice that an assignment operator assigns the value returned from the File.CreateText method to the outputFile variable. This causes the outputFile variable to reference the StreamWriter object that was created by the method.

Writing Data to a File with a StreamWriter Object

- After the statement in line 2 executes, the courses.txt file is created on the disk, a StreamWriter object associated with the file exists in memory, and the outputFile variable references that object.
- **Line 4:** This statement writes the string "Intro to Computer Science" to the courses. txt file. It does that by calling the StreamWriter class's WriteLine method, passing the string that is to be written to the file as an argument. When the WriteLine method writes data to a file, it writes a newline character immediately following the data. A newline character is an invisible character that specifies the end of a line of text.
- **Line 5:** This statement writes the string "English Composition" to the courses.txt file.
- **Line 6:** This statement writes the string "Calculus I" to the courses.txt file.
- **Line 7:** This statement writes the string "Music Appreciation" to the courses.txt file.
- **Line 9:** This statement closes the courses.txt file. It does that by calling the StreamWriter class's Close method.

Writing Data to a File with a StreamWriter Object

- After the code is executed, we can open the courses.txt file using a text editor and look at its contents.
- The Figure below show how the file's contents will appear in Notepad.

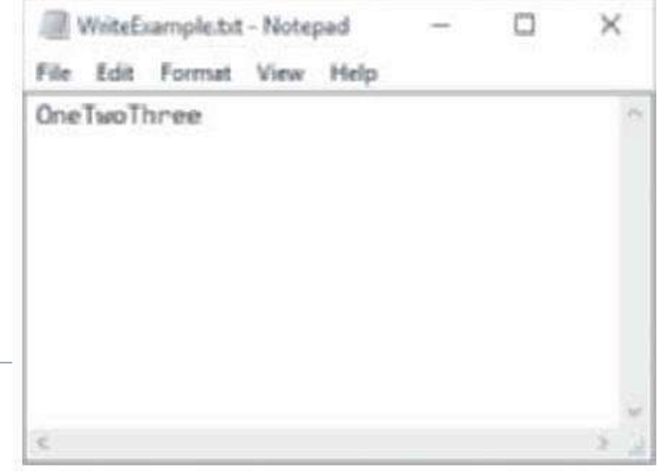


Writing Data with the Write Method

- The StreamWriter class provides the Write method: It writes an item of data to a text file without writing a newline character.
- For example, the following code sample opens a file named Example.txt and then uses the WriteLine method to write the strings "One", "Two", and "Three" to the file.

```
1 StreamWriter outputFile;
2 outputFile = File.CreateText("Example.txt");
3
4 outputFile.Write("One");
5 outputFile.Write("Two");
6 outputFile.Write("Three");
7
8 outputFile.Close();
```

- The resulting file in Notepad:



Handling File-Related Exceptions

- Unexpected problems can potentially occur when working with files.
- For example, your program might not have sufficient rights to create a file when it calls the File. CreateText method, or the disk might be full when you call the StreamWriter class's WriteLine method.
- To handle such exceptions, you can write a try-catch statement

```
try
{
    StreamWriter outputFile;
    outputFile = File.CreateText("courses.txt");

    outputFile.WriteLine("Intro to Computer Science");
    outputFile.WriteLine("English Composition");
    outputFile.WriteLine("Calculus I");
    outputFile.WriteLine("Music Appreciation");

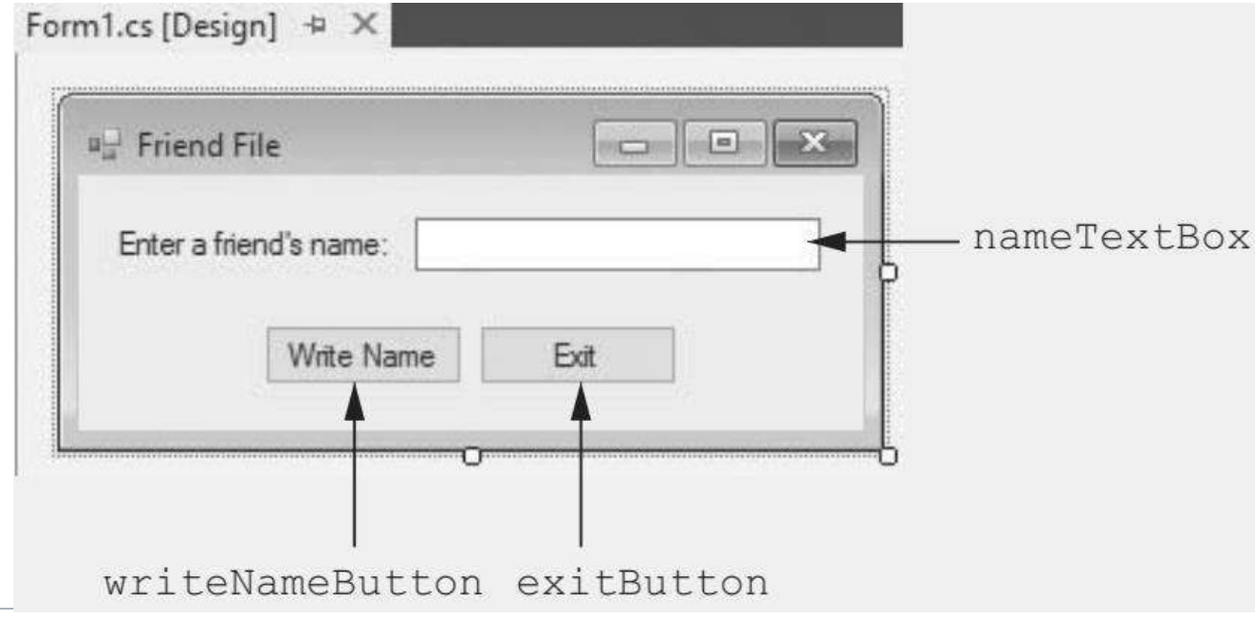
    outputFile.Close();
}
catch (Exception ex)
{
    // Display an error message.
    MessageBox.Show(ex.Message);
}
```

Example 6

Friend File application

Create an application that allow the user to enter a name into the nameTextBox control.

When the user clicks the writeNameButton control, the application opens a text file named Friend.txt, writes the name that was entered into the TextBox control to the file, and then closes the file.



Example 6- Form1 code for Friend File application

```
using System.IO;

namespace Friend_File
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }

        private void writeNameButton_Click(object sender, EventArgs e
        {
            try
            {
                // Declare a StreamWriter variable.
                StreamWriter outputFile;

                // Create a file and get a StreamWriter object.
                outputFile = File.CreateText("Friend.txt");

                // Write the friend's name to the file.
                outputFile.WriteLine(nameTextBox.Text);

                // Close the file.
                outputFile.Close();

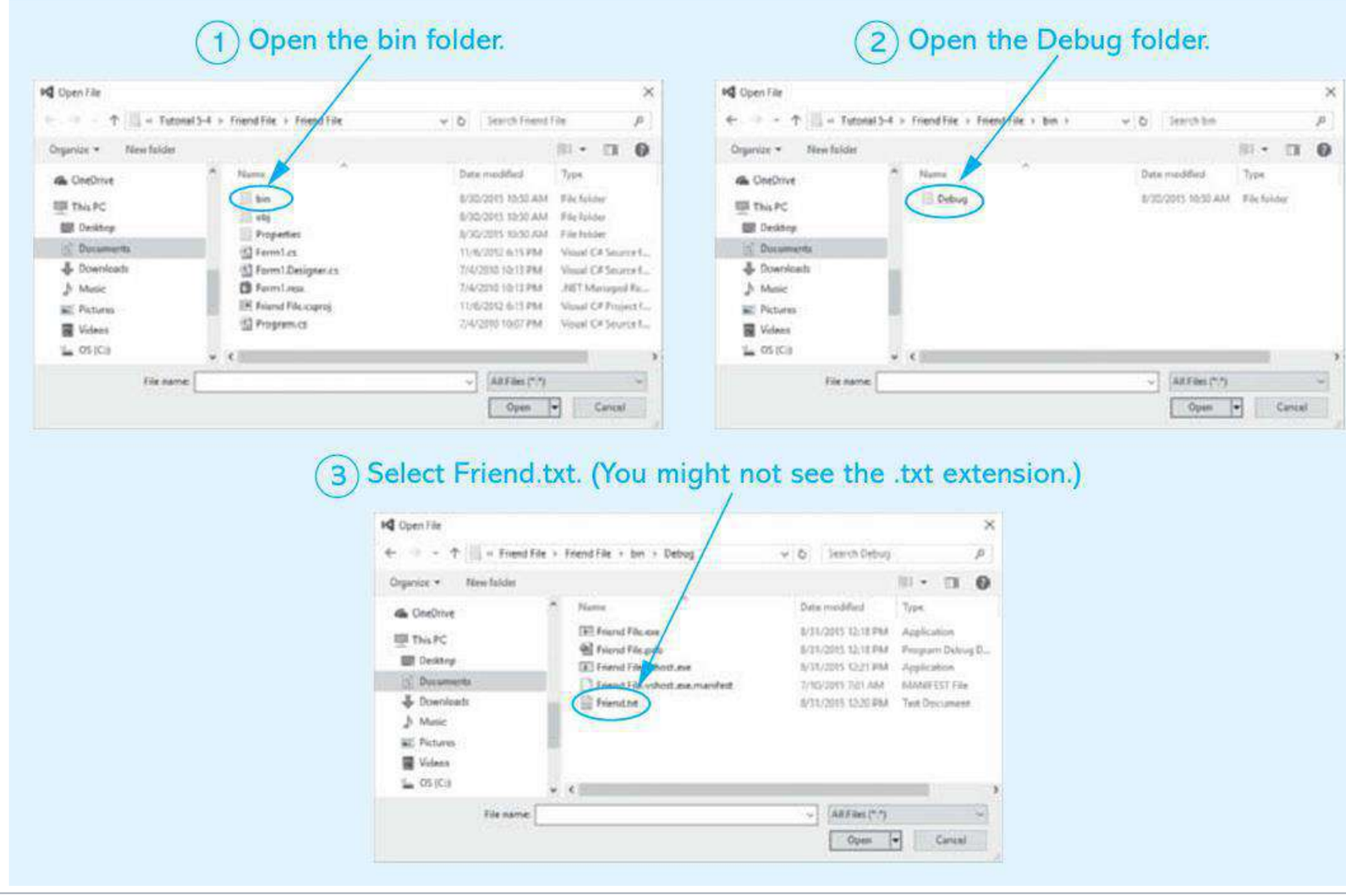
                 // Let the user know the name was written.
                 MessageBox.Show("The name was written.");
            }
            catch (Exception ex)
            {
                 // Display an error message.
                 MessageBox.Show(ex.Message);
            }
        }

        private void exitButton_Click(object sender, EventArgs e)
        {
            // Close the form.
            this.Close();
        }
    }
}
```

Example 6- Opening the Friend .txt file in the Open File window

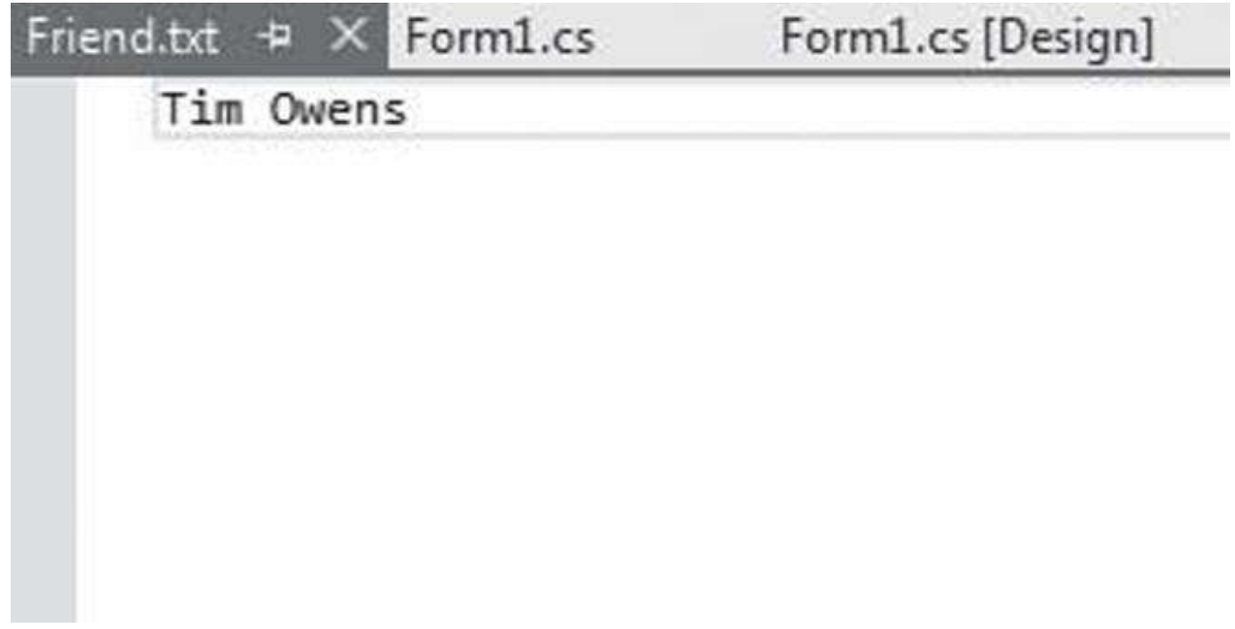
- Save the project. Then, press F5 on the keyboard to compile and run the application.
- Enter a name into the nameTextBox control, and then click the Write Name button. You should see a message box appear letting you know that the name was written to the file. Click the OK button to dismiss the message box; then click the Exit button on the application's form to end the application.
- Now you will look at the contents of the Friend.txt file that the application created. Click File on the menu bar, then click Open, and then click File. . .
- You should now see the Open File window, viewing the contents of the Friend File project folder. As shown in the next Figure, open the bin folder, then open the Debug folder, and then select the file Friend.txt. Click the Open button.

Example 6- Opening the Friend .txt file in the Open File window



Example 6- contents of the Friend .txt file shown in Visual Studio

- The contents of the Friend.txt file in Visual Studio shown in the Figure. (The example in the figure shows the contents of the file after the user has written Tim Owens to the file.)



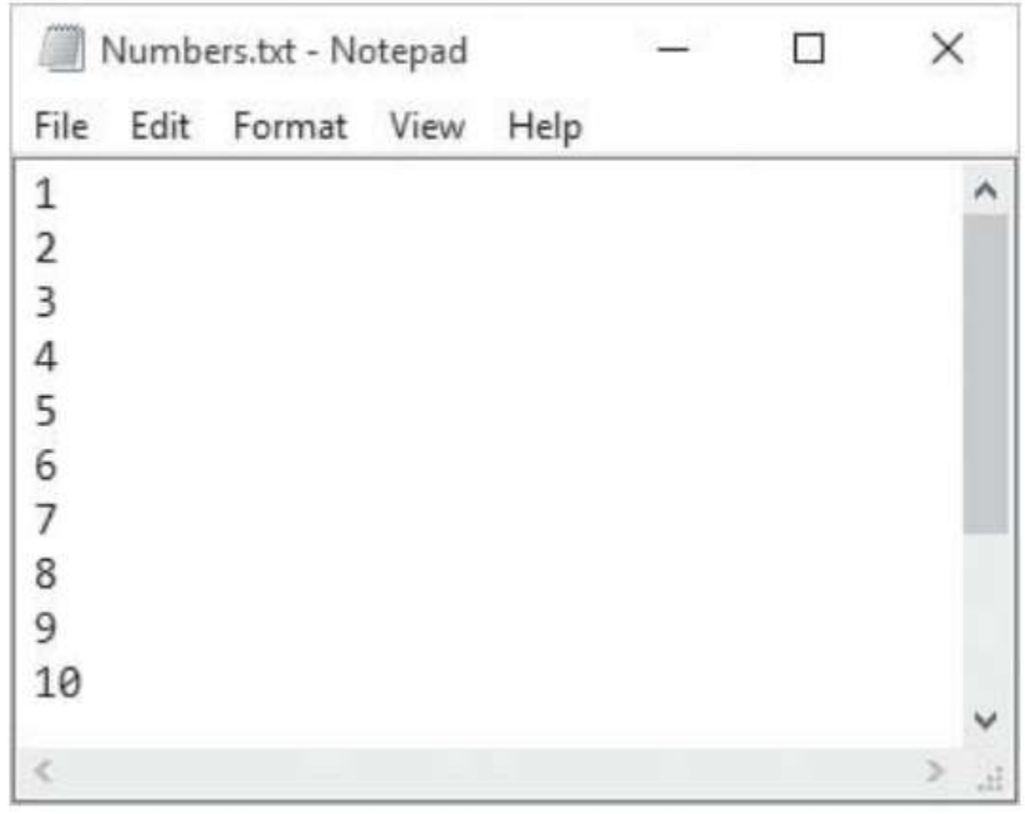
Writing Numeric Data to a Text File

- You can use the StreamWriter class's WriteLine or Write method to write numbers (such as ints, doubles, and decimals) to a text file, but the numbers are converted to strings.
- **Example:**

```
1  private void writeNumbersButton_Click(object sender, EventArgs e)
2  {
3      try
4      {
5          // Declare a StreamWriter variable.
6          StreamWriter outputFile;
7
8          // Create a file and get a StreamWriter object.
9          outputFile = File.CreateText("Numbers.txt");
10
11         // Write the numbers 1 through 10 to the file.
12         for (int count = 1; count <= 10; count++)
13         {
14             outputFile.WriteLine(count);
15         }
16
17         // Close the file.
18         outputFile.Close();
19     }
20     catch (Exception ex)
21     {
22         // Display an error message.
23         MessageBox.Show(ex.Message);
24     }
25 }
```

Writing Numeric Data to a Text File

- The Figure below shows how the file appears when opened with Notepad.

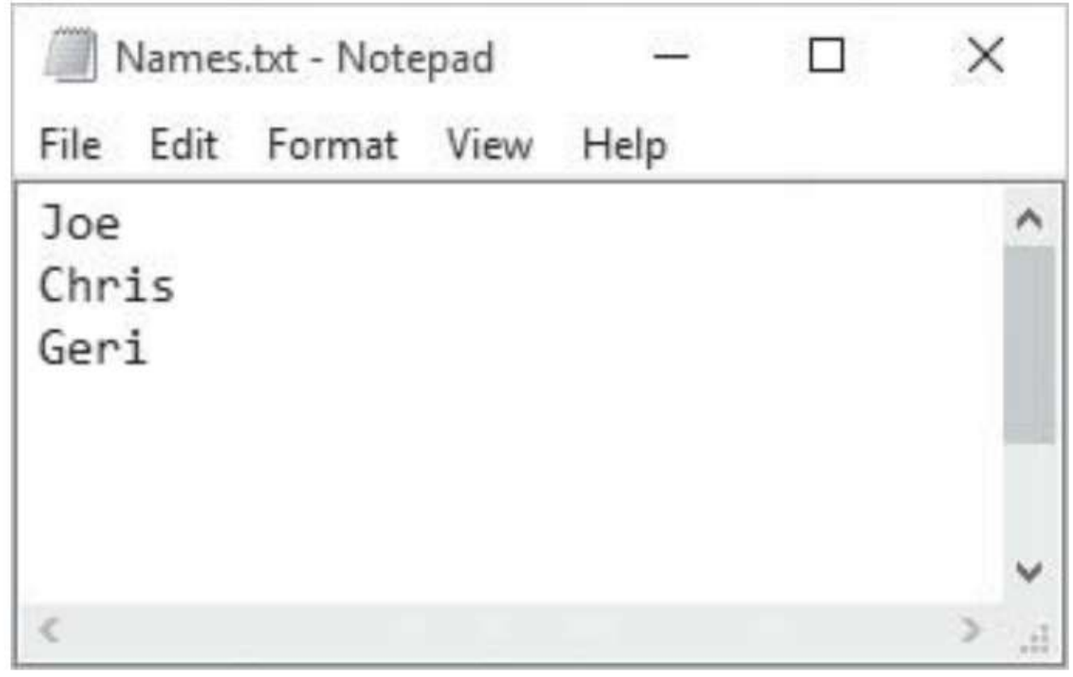


Appending Data to an Existing File

- When you call the File.CreateText method to open a file and the file specified by the argument already exists, it is erased and a new empty file with the same name is created.
- For example, when you run the Friend File application, each time you click the Write Name button, the Friend.txt file is erased and a new file is created.
- Sometimes you want to preserve an existing file and append new data to its current contents.
- To append data to an existing file, you open it with the File.AppendText method.
- It works like the File.CreateText method, but the file is not erased if it already exists.
- Any data written to the file is appended to the file's existing contents.

Example 7

- Assume the file Names.txt exists and contains the data shown:

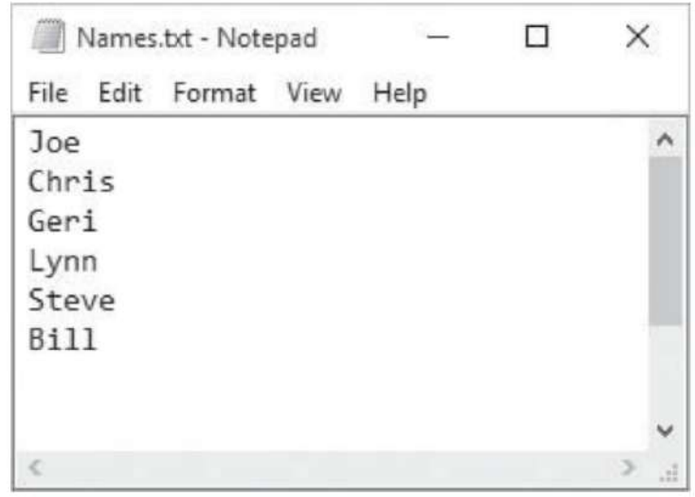


Example 7

- The following code opens the file and appends additional data to its existing contents

```
1 StreamWriter outputFile;
2 outputFile = File.AppendText("Names.txt");
3
4 outputFile.WriteLine("Lynn");
5 outputFile.WriteLine("Steve");
6 outputFile.WriteLine("Bill");
7
8 outputFile.Close();
```

- After this code executes, the Names.txt file contains the data shown in the Figure:



Example 8

Appending Data to the Friend .txt File

Modify the Friend File application so it appends data to the Friend.txt file.

When the user clicks the Write Name button, instead of erasing the file's current contents, the application adds the contents of the nameTextBox control to the Friend.txt file.

Example 8- code for Form1 in the Friend File application

```
private void writeNameButton_Click(object sender, EventArgs e)
{
    try
    {
        // Declare a StreamWriter variable.
        StreamWriter outputFile;

        // Open the Friend.txt file for appending,
        // and get a StreamWriter object.
        outputFile = File.AppendText("Friend.txt");

        // Write the friend's name to the file.
        outputFile.WriteLine(nameTextBox.Text);

        // Close the file.
        outputFile.Close();

        // Let the user know the name was written.
        MessageBox.Show("The name was written.");

        // Clear the nameTextBox control.
        nameTextBox.Text =  "";

        // Give the focus to the nameTextBox control.
        nameTextBox.Focus();
    }
    catch (Exception ex)
    {
        // Display an error message.
        MessageBox.Show(ex.Message);
    }
}
```

Specifying the Location of an Output File

- When you call the File.CreateText or File.AppendText methods to open a file, you pass the filename as a string argument.
- If the filename that you pass as an argument does not contain the file's path, the file's location will be the bin\Debug folder, under the application's project folder.
- you can specify a path as well as a filename in the argument that you pass to the File.CreateText or File.AppendText method.
- **Example**

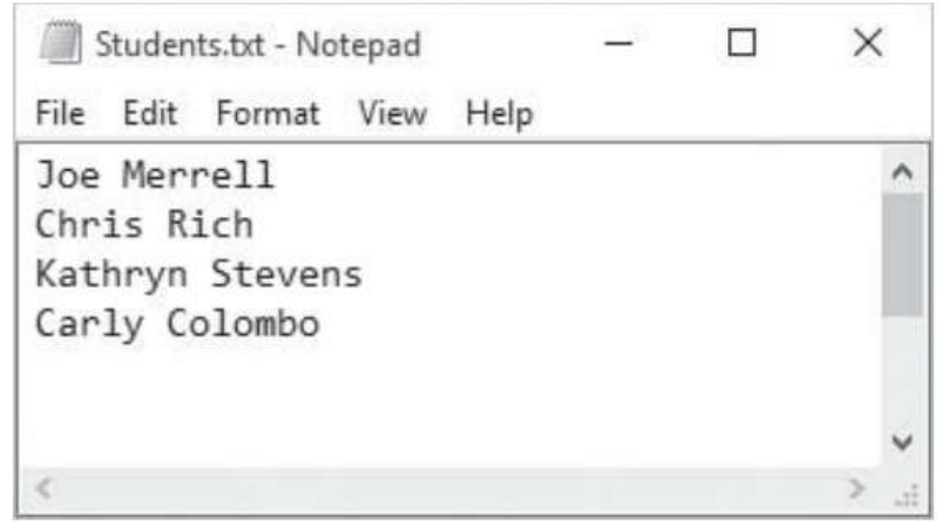
```
StreamWriter outputFile;
outputFile = File.CreateText(@"C:\Users\Chris\Documents\Names.txt");
```

Reading Data from a File with StreamReader Object

- To read data from a text file, you create a StreamReader object.
- You can then use the StreamReader class's ReadLine method to read a line of text from a file.

Example 9

Suppose a file named Students.txt exists and contains the four names as shown



Write a program that reads the four names from the Students.txt file and displays them in message boxes.

Example 9- code

```
try
{
   // Declare a variable to hold an item read from the file.
   string studentName;

   // Declare a StreamReader variable.
   StreamReader inputFile;

   // Open the file and get a StreamReader object.
   inputFile = File.OpenText("Students.txt");

    // Read and display the first name.
    studentName = inputFile.ReadLine();
    MessageBox.Show(studentName);

    // Read and display the second name.
    studentName = inputFile.ReadLine();
    MessageBox.Show(studentName);

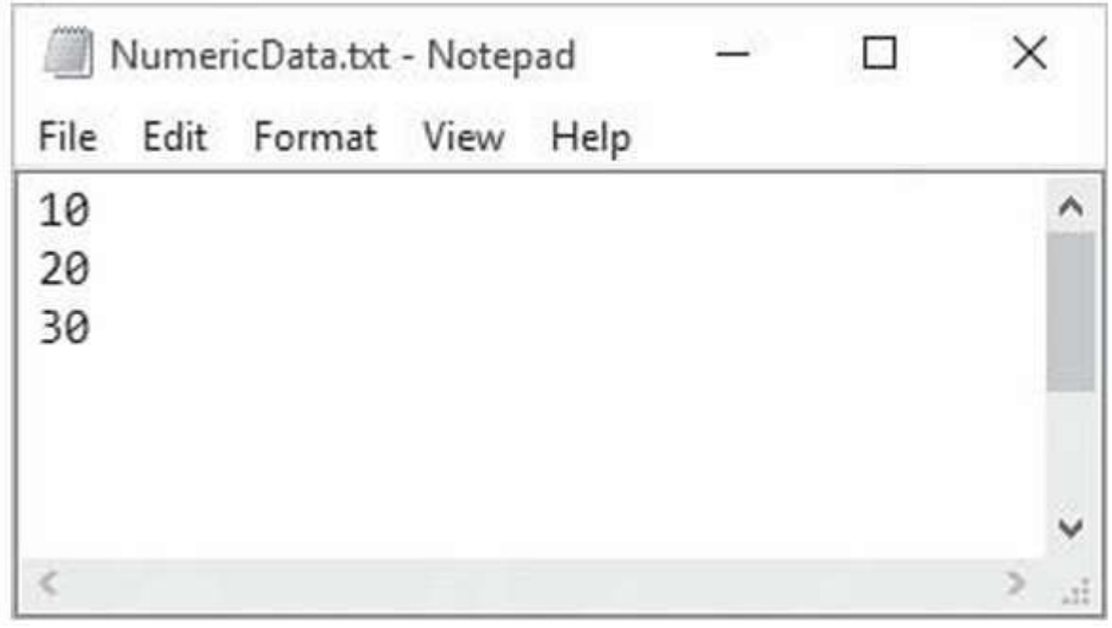
    // Read and display the third name.
    studentName = inputFile.ReadLine();
    MessageBox.Show(studentName);

    // Read and display the fourth name.
    studentName = inputFile.ReadLine();
    MessageBox.Show(studentName);

    // Close the file.
    inputFile.Close();
}
catch (Exception ex)
{
    // Display an error message.
    MessageBox.Show(ex.Message);
}
```

Reading Numeric Data from a Text File

- Suppose a text file contains numeric data, such as that shown in the Figure. When we use the ReadLine method to read the items from the file, we get the strings "10", "20", and "30".

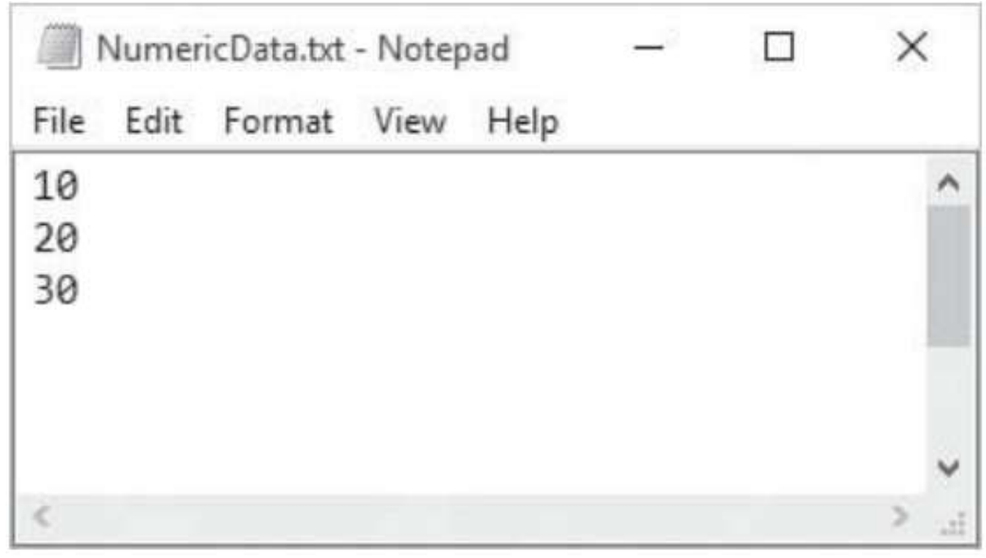


- If we need to perform math with these values, we must convert each value from a string to a numeric data type.

- We can use the Parse or TryParse families of methods to perform this conversion.

Example 10

Write a program that reads the three numbers from the NumericData.txt file shown in the Figure and displays their total in a message box.



Example 10- Code

```
try
{
    // Variables to hold the numbers read from the file
    // and their total
    int number1, number2, number3, total;

    // A StreamReader variable.
    StreamReader inputFile;

    // Open the file and get a StreamReader object.
    inputFile = File.OpenText("NumericData.txt");

    // Read three numbers from the file.
    number1 = int.Parse(inputFile.ReadLine());
    number2 = int.Parse(inputFile.ReadLine());
    number3 = int.Parse(inputFile.ReadLine());

    // Calculate the total of the numbers.
    total = number1 + number2 + number3;

    // Display the total.
    MessageBox.Show("The total is " + total);

    // Close the file.
    inputFile.Close();
}
catch (Exception ex)
{
    // Display an error message.
    MessageBox.Show(ex.Message);
}
```

Reading a File with a Loop and Detecting the End of the File

- A program must read the contents of a file without knowing the number of items that are stored in the file.
- However, an exception will be thrown if the program attempts to read beyond the end of the file.
- The program needs some way of knowing when the end of the file has been reached so it will not try to read beyond it.
- StreamReader objects have a Boolean property named EndOfStream that signals whether the end of the file has been reached.
- If the file's read position is at the end of the file (and there is no more data to read), the EndOfStream property is set to true. Otherwise, it is set to false.

Reading a File with a Loop and Detecting the End of the File

- Assume inputFile references a StreamReader object that is associated with a file that is already open.

```
while (inputFile.EndOfStream == false)
{
   // Read an item from the file.
   // Process the item.
}
```

- However, most programmers prefer the following logic, which uses the ! operator:

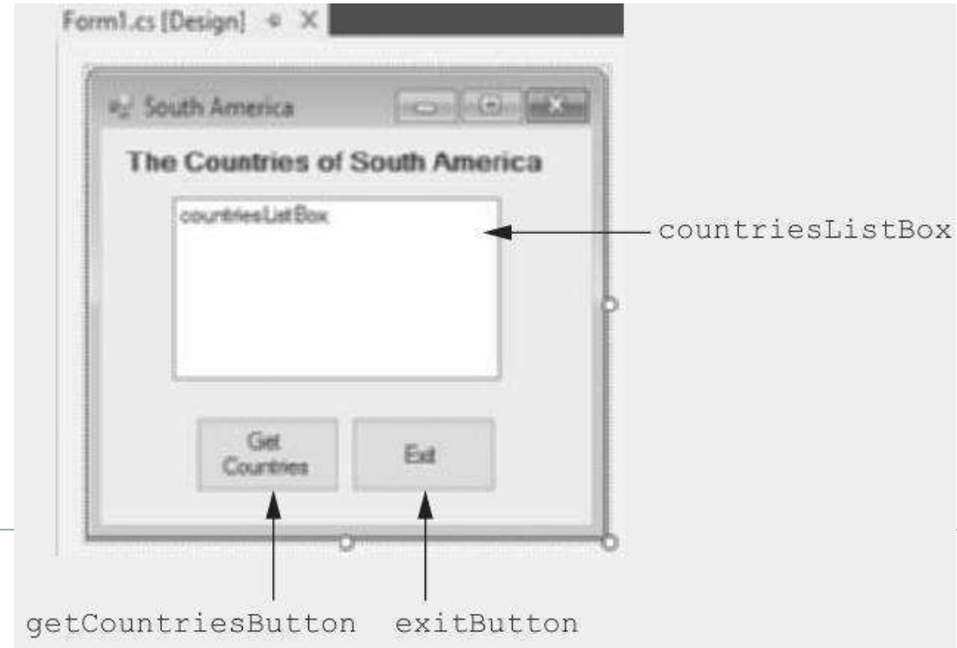
```
while (!inputFile.EndOfStream)
{
   // Read an item from the file.
   // Process the item.
}
```

Example 11

South America Application

The application has an accompanying text file named Countries.txt that is stored in the bin\Debug folder under the project folder. The Countries.txt file contains the names of the countries of South America.

When the application runs and the user clicks the Get Countries button, the application reads each country name from the file and adds each one to the countriesListBox control.



Example 11- Code

```
using System.IO;

namespace South_America
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }

        private void getCountriesButton_Click(object sender, EventArgs e)
        {
            try
            {
                // Declare a variable to hold a country name.
                string countryName;

                // Declare a StreamReader variable.
                StreamReader inputFile;

                // Open the file and get a StreamReader object.
                inputFile = File.OpenText("Countries.txt");

                // Clear anything currently in the ListBox.
                countriesListBox.Items.Clear();

                // Read the file's contents.
                while (!inputFile.EndOfStream)
                {
                    // Get a country name.
                    countryName = inputFile.ReadLine();

                    // Add the country name to the ListBox.
                    countriesListBox.Items.Add(countryName);
                }

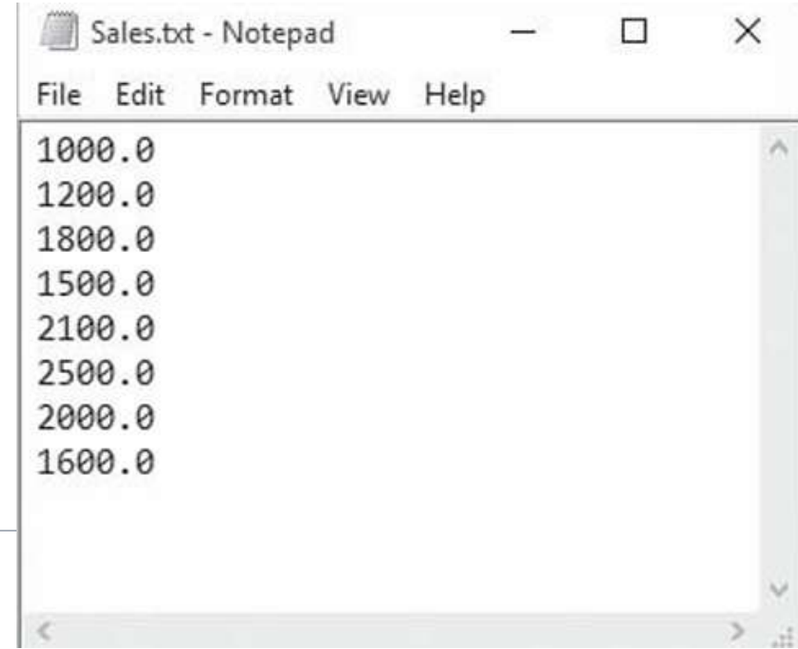
                // Close the file.
                inputFile.Close();
            }
            catch (Exception ex)
            {
                // Display an error message.
                MessageBox.Show(ex.Message);
            }
        }
```

Example 12

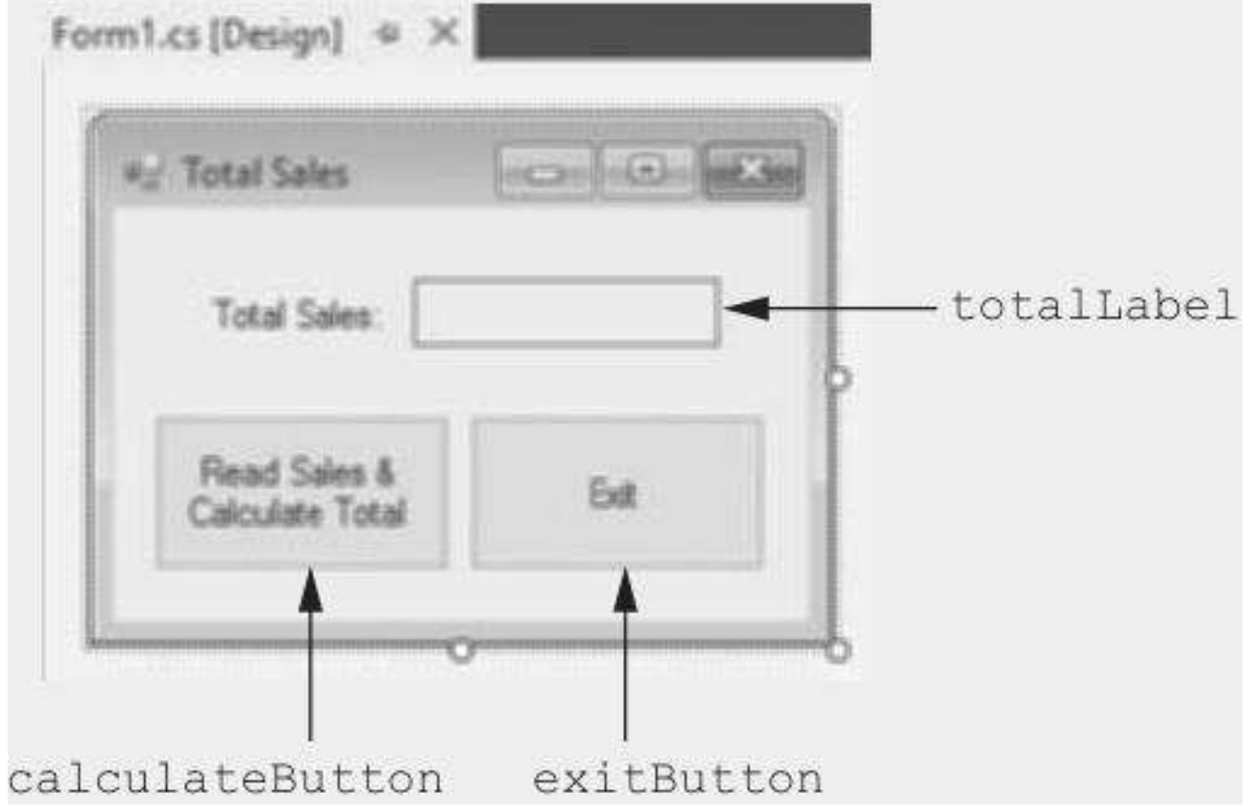
Total Sales Application

The application has an accompanying text file named Sales.txt that is stored in the bin\Debug folder, under the project folder. The Sales. txt file contains the amounts shown in the Figure.

When the completed application runs and the user clicks the Read Sales & Calculate Total button, the application calculates the total of the values in the Sales.txt file and displays the total in the totalLabel control.



Example 12- application form



Example 12- Code

```
private void calculateButton_Click(object sender, EventArgs e)
{
    try
    {
       // Variables
       decimal sales;        // To hold a sales amount
       decimal total = 0m;   // Accumulator, set to 0

       // Declare a StreamReader variable.
       StreamReader inputFile;

       // Open the file and get a StreamReader object.
       inputFile = File.OpenText("Sales.txt");

       // Read the file's contents.
       while (!inputFile.EndOfStream)
       {
           // Get a sales amount.
           sales = decimal.Parse(inputFile.ReadLine());

           // Add the sales amount to total.
           total += sales;
       }

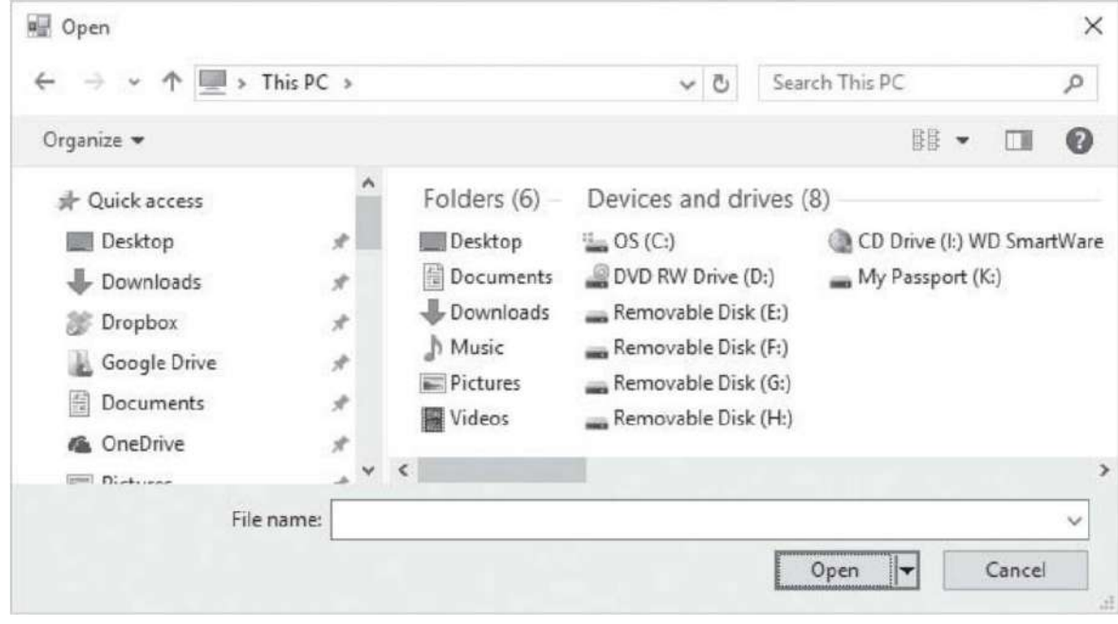
       // Close the file.
       inputFile.Close();

       // Display the total.
       totalLabel.Text = total.ToString("C");

    }
    catch (Exception ex)
    {
        // Display an error message.
        MessageBox.Show(ex.Message);
    }
}
```

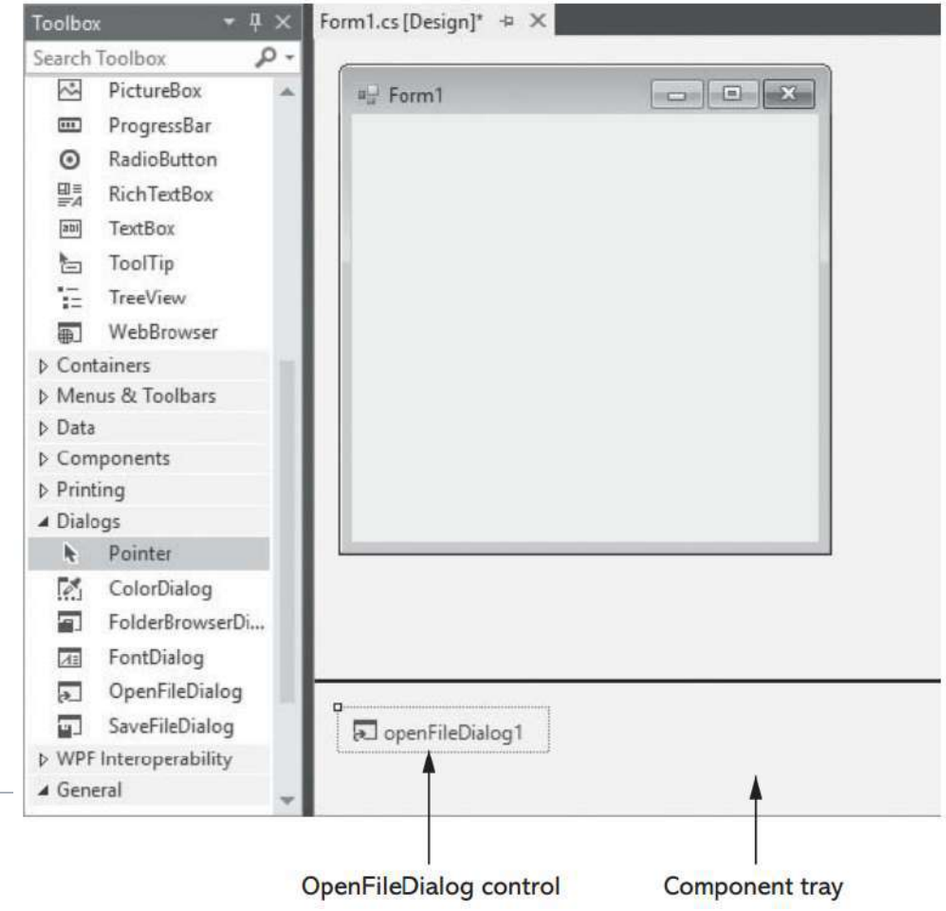
OpenFileDialog Control

- The OpenFileDialog control displays a standard Windows Open dialog box, such as the one shown in the Figure below.
- The Open dialog box is useful in applications that must open an existing file because it allows the user to browse the system and select the file.



OpenFileDialog Control

- To add an OpenFileDialog control to a form, double-click the OpenFileDialog tool under the Dialogs group in the Toolbox window.
- When the control is created, it does not appear on the form, but in an area at the bottom of the Designer known as the component tray.



Displaying an Open Dialog Box

- In code, you can display an Open dialog box by calling the OpenFileDialog control's ShowDialog method:

```
openFile.ShowDialog();
```

assume that we have created an OpenFileDialog control and changed its name to openFile.

- In most cases, however, you will want to know whether the user clicked the Open button or the Cancel button to dismiss the Open dialog box.
  - If the user clicked the Open button, it means that the user has selected a file and he or she can open it.
  - If the user clicked the Cancel button, it means that the user does not want to proceed.

Displaying an Open Dialog Box

- The ShowDialog method returns a value that indicates which button the user clicked to dismiss the dialog box.
  - If the user clicked the Open button, the value DialogResult.OK is returned.
  - If the user clicked the Cancel button, the value DialogResult.Cancel is returned.

- Example:

```
if (openFile.ShowDialog() == DialogResult.OK)
{
   MessageBox.Show("You clicked the Open button.");
}
else
{
   MessageBox.Show("You clicked the Cancel button.");
}
```

Filename Property

- When the user selects a file with the Open dialog box, the file's path and filename are stored in the control's Filename property.
- Assume openFile is the name of an OpenFileDialog control. The following code is an example of how you can display an Open dialog box and, if the user clicks the Open button to dismiss the dialog box, open the selected file:

```
1  StreamReader inputFile;
2
3  if (openFile.ShowDialog() == DialogResult.OK)
4  {
5     // Open the selected file.
6     inputFile = File.OpenText(openFile.Filename);
7
8     // Continue processing the file…
9  }
10 else
11 {
12    MessageBox.Show("Operation canceled.");
13 }
```

InitialDirectory Property

- By default, the Open dialog box displays the contents of the user's Documents directory (or folder).
- You can specify another directory to be initially displayed by storing its path in the InitialDirectory property.
- For example, the following code stores the path C:\Data in the openFile control's InitialDirectory property before displaying an Open dialog box:

```
openFile.InitialDirectory = "C:\Data";

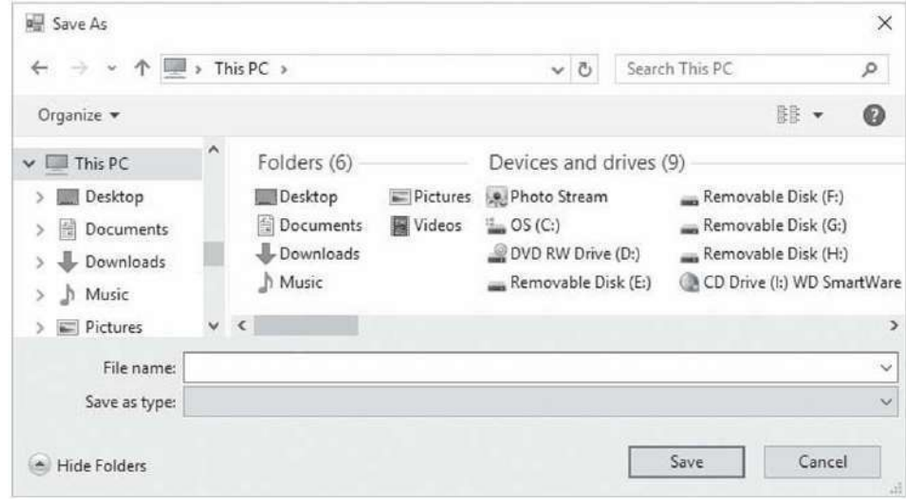
if (openFile.ShowDialog() == DialogResult.OK)
{
    // Continue to process the selected file…
}
else
{
    // The operation was canceled.
}
```

Title Property

- By default, the word Open is displayed in an Open dialog box's title bar.
- You can change the default text displayed in the title bar by changing the control's Title property.

SaveFileDialog Control

- The SaveFileDialog control displays a standard Windows Save As dialog box, such as the one shown in the Figure



- To add a SaveFileDialog control to a form, double-click the SaveFileDialog tool under the Dialogs group in the Toolbox window.
- When the control is created, it appears in the component tray at the bottom of the Designer.
- The control will be given a default name such as saveFileDialog1, but you can change the name with the Name property.

Display a Save As Dialog Box

- In code, you can display a Save As dialog box by calling the SaveFileDialog control's ShowDialog method.
- For example, assume that we have created a SaveFileDialog control and changed its name to saveFile.
- The following statement calls the control's ShowDialog method:

```
saveFile.ShowDialog();
```

- The method returns a value indicating whether the user clicked the Save button or the Cancel button to dismiss the Save As dialog box.
  - If the user clicks the Save button, the value DialogResult.OK is returned.
  - If the user clicks the Cancel button, the value DialogResult.Cancel is returned.
- Example:

```
if (saveFile.ShowDialog() == DialogResult.OK)
{
   MessageBox.Show("You clicked the Save button.");
}
else
{
   MessageBox.Show("You clicked the Cancel button.");
}
```

Filename Property

- When the user specifies a location and filename with the Save As dialog box, the file's path and filename are stored in the control's Filename property.
- Assume saveFile is the name of a SaveFileDialog control. The following code is an example of how you can display a Save As dialog box, and if the user clicks the Save button to dismiss the dialog box, open the selected file:

```
1  StreamWriter outputFile;
2
3  if (saveFile.ShowDialog() == DialogResult.OK)
4  {
5     // Create the selected file.
6     outputFile = File.CreateText(saveFile.Filename);
7
8     // Write data to the file…
9  }
10 else
11 {
12     MessageBox.Show("Operation canceled.");
13 }
```

InitialDirectory Property

- By default, the Save As dialog box displays the contents of the user's Documents directory (or folder).
- You can specify another directory to be initially displayed by storing its path in the InitialDirectory property.
- The following code stores the path C:\Data in the saveFile control's InitialDirectory property before displaying a Save As dialog box:

```
saveFile.InitialDirectory = "C:\Data";

if (saveFile.ShowDialog() == DialogResult.OK)
{
   // Continue to process the file…
}
else
{
   // The operation was cancelled.
}
```

Title Property

- By default the words Save As are displayed in a Save As dialog box's title bar.
- You can change the default text displayed in the title bar by changing the control's Title property.